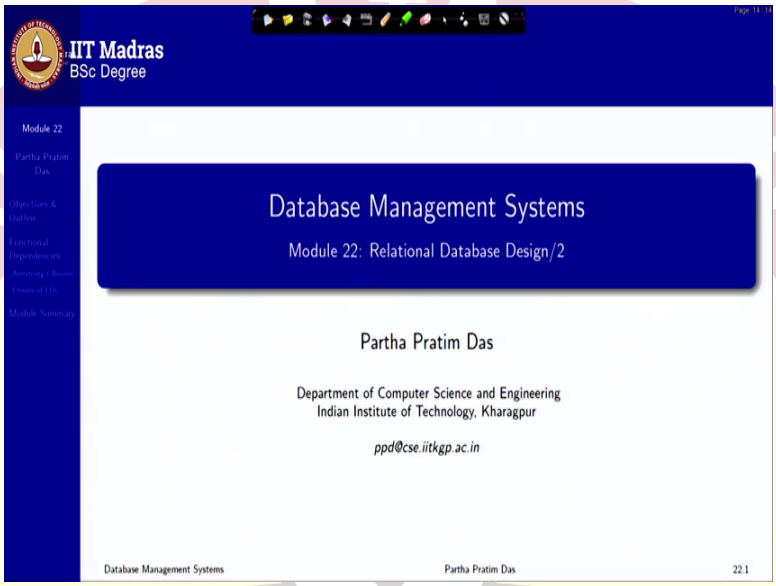
IIT Madras

ONLINE DEGREE

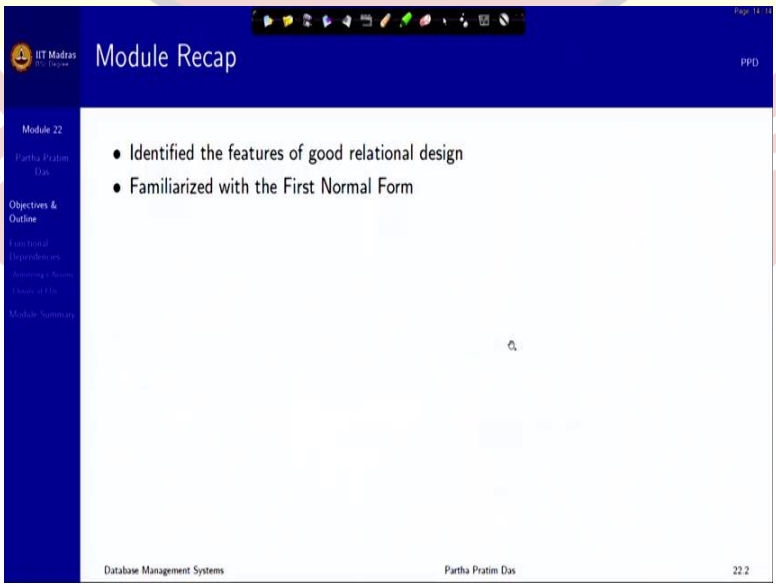
Database Management Systems
Professor Partha Pratim Das
Department of Computer Science & Engineering
Indian Institute of Technology, Kharagpur
Relational Database Design/2

(Refer Slide Time: 00:20)



Welcome to Module 22 of Database Management Systems course in the IIT Madras online B.Sc. program.

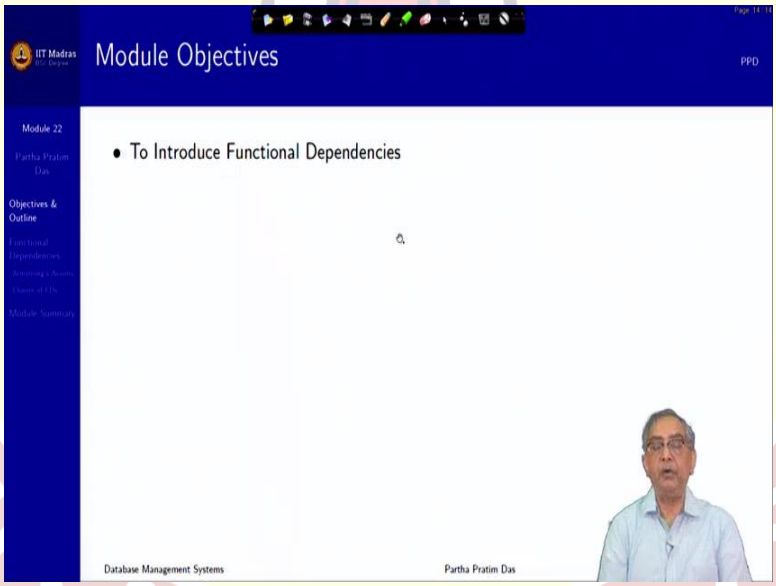
(Refer Slide Time: 00:25)

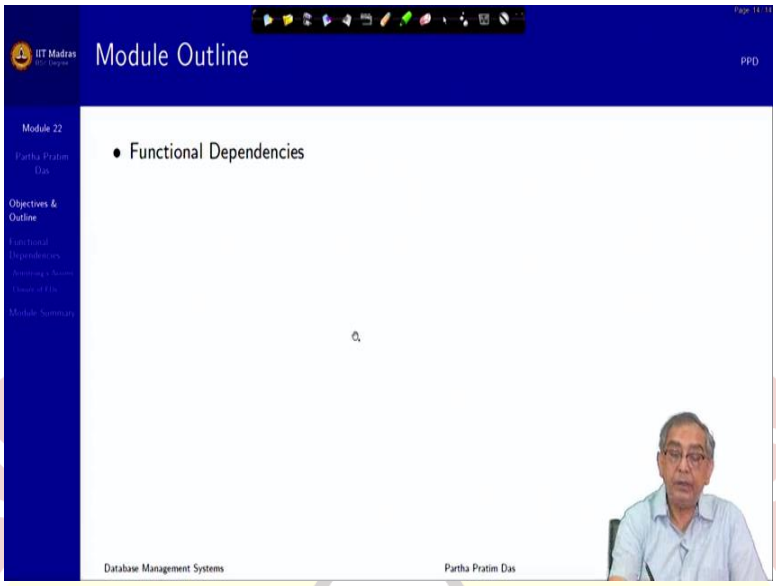


In the last module, we identified the features of good relational design, because we have got started into actually formally doing the relational design of the relational schemas and we talked about the various factors like anomaly, redundancy, decomposition and normalization to see, to understand that we need to normalize the schemas to effectively reduce the redundancy and the anomaly that can happen.

We also looked at atomic domains and the fact that every relational schema must add here to certain normal form, certain, it must satisfy certain properties and we saw in that the first normal form. Every relational schema must be in that. It requires that every attribute have a domain which is not composite, which is atomic and no attribute be multiple valued. And through an example we saw that if a relationship is not in one normal form, because of certain one to many relationship within the atributes then it can be easily decomposed into one.

(Refer Slide Time: 01:44)



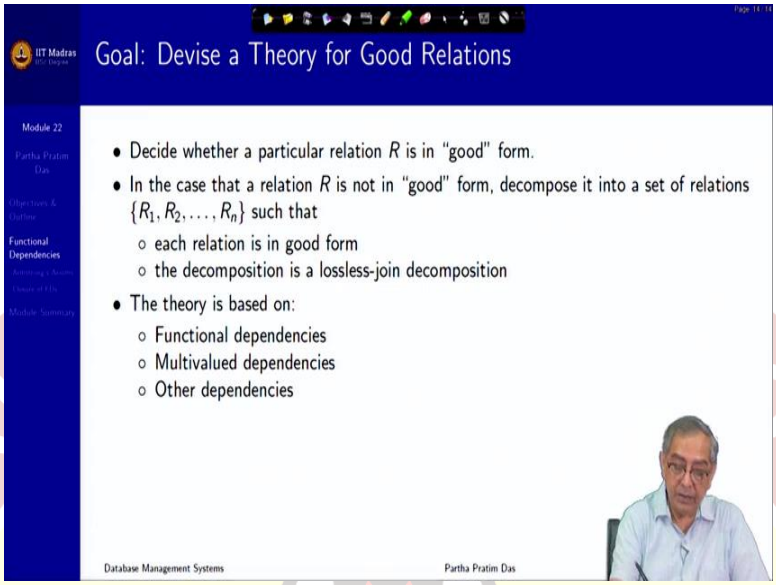


So, in the process we mentioned about functional dependency. We will formally introduce it now and take that forward.

(Refer Slide Time: 02:00)

So, in today's module we are going to talk about functional dependencies.

(Refer Slide Time: 02:04)

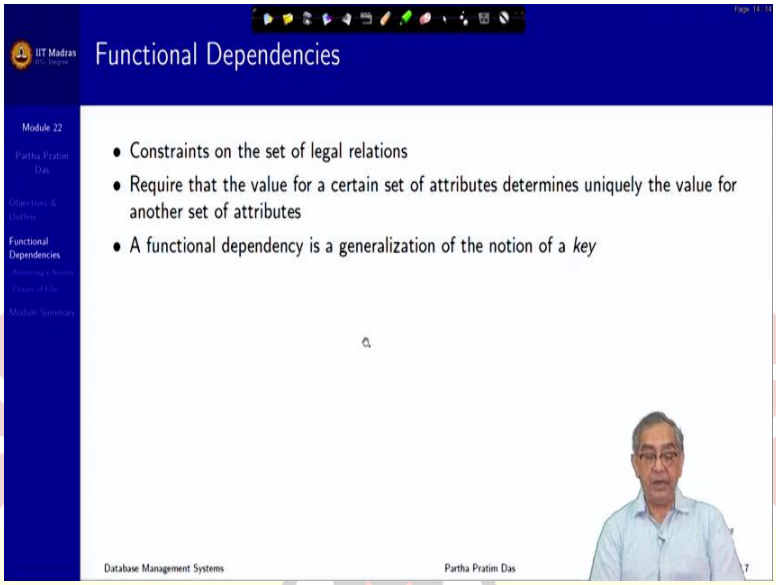


So, why we are studying functional dependencies? Because we want to develop a formal mathematical theory for good relations. It cannot just be on judgment. It is not manageable to do everything by hand, because in a reasonable database application, you do not have two, three schemas. You will have 50, 100, 200, even more number of schemas covering various aspects of the problem domain.

So, we say that in a relation R is not in good form then we must decompose it into a set of relations R1 to Rn, where n is 2 or more so that every relation is in good form. Naturally, that is why we are decomposing. So, what is the point decomposing if we create a decomposed relation which itself is not good. And the decomposition is lossless join. We have seen this in the last module that it must be possible to take a natural join and get back the original relation for every instance.

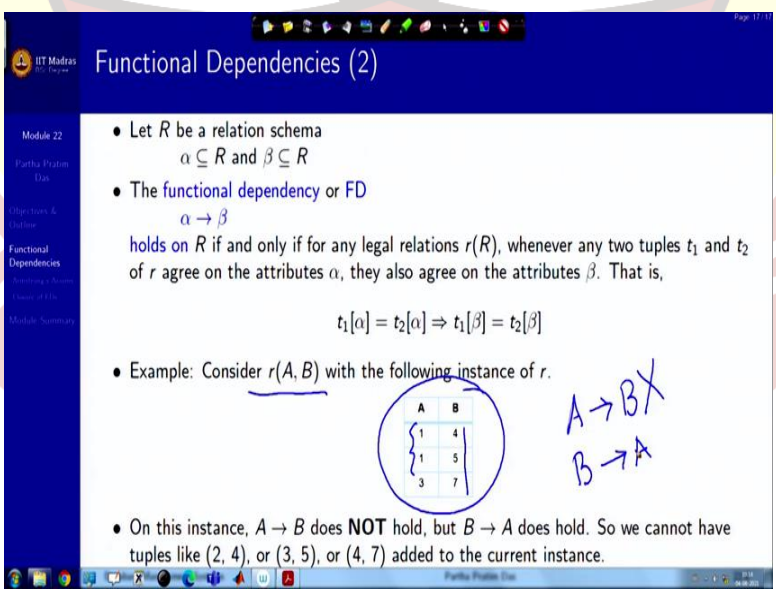
So, this theory is based on various types of dependencies, starting with the functional dependency, then we have multi-valued **dependencie**s and other dependencies. We are now talking about functional dependencies.

(Refer Slide Time: 03:40)



So, these are constraints on the legal set of relations which require the value of a certain set of attributes determining uniquely the value of another set of attributes. We saw just a glimpse. Department name uniquely decides what is the budget and what is the building. So, functional dependencies in that way is a generalization of the notion of key.

(Refer Slide Time: 04:02)



So, let us look at functional dependency in formal terms. So, I have a relational schema R and two subsets of it alpha and beta. A functional dependency or FD is written as alpha determines beta. This will hold on the relation R if and only if both ways. For any legal instance of the

schema R, which we denote as small r, whenever two tuples t1 and t2 of R agree on the attribute alpha, they must agree on that tribute beta.

So, department name **uniquely or functionally** determines budget and building, which means, if I have two tuples whose department name is same, then their budget has to be same, then their building has to be same, which means they have to be identical tuples, which obviously are not allowed.

So, just think about this very carefully. So, what we mean by t1 alpha is, if I have the entire tuple t and this is my set of **attributes alpha** and this is my set of attributes beta. I am sorry, let us draw it in a cleaner way. This is where, so I have a set of attributes alpha, I have a set of attributes beta, there may be some overlaps, I have t.

Now, by t alpha, we mean a tuple which just takes the values of t1 alpha attributes. Similarly, this is this part of what you get here will be t beta. So, using that notation, we say that, we have two tuples t1 and t2. They have the same alpha values that is there. Alpha part tuple of t1 and alpha part tuple of t2 are same. If that happens, then their beta parts will also have to be same. When that happens for all possible instances currently present was there in the past and will be there in the future which you may not know but has to happen, then we say this functional dependency alpha determines beta holds.

It is not based on one instance just or couple of instances. It has to be for all instances. Only then the dependency holds as in the case of department name, determining budget and building. This just gives you the basic principle **or basic rules**, business rules that the real world has. Now, why is it that department cannot have two buildings, it can. In several universities the departments belongs in five buildings. If that is the case, then we will not have department functionally determining building.

But under the business rule given to us by the client, the client has said that no department will be in more than one building. So, we can, from that we are forming the functional dependencies, not from the data, because all possible data we do not know. So, that rule, that business rule, that principle tells us that if I know the department, I will certainly know the building because it can exist only in one building. And it uniquely determines that building, therefore, department functionally determines the building. A similar statement can be made for budget.

Department may have one budget or some organization, say we have multiple budgets on this head, on that head, on that head and so on. In that case, this functional dependency will not work. So, it is basically, functional dependencies are encoding of constraints, business rules that exist in the real world. They are not just instances of the data. They have to hold on the instances of the data.

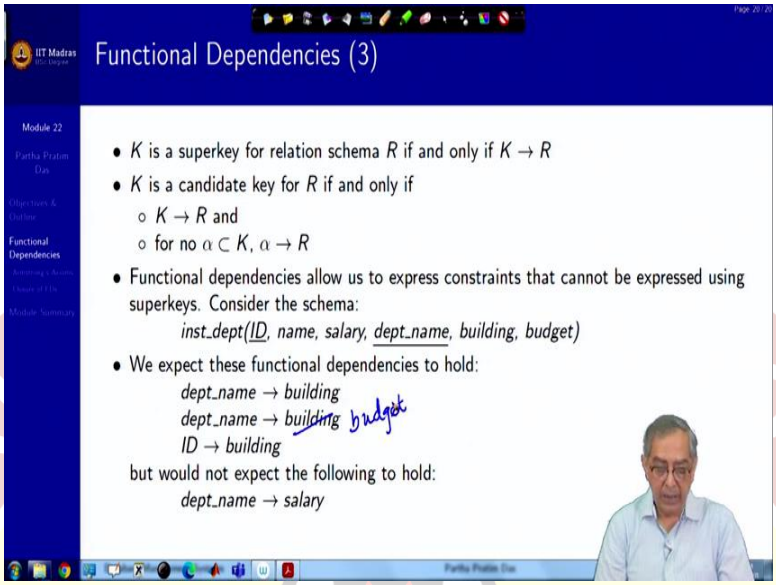
So, to take a simple example, let us consider a two attribute relationship and there is a instance of this schema, you can very easily see that I have two tuples 1, 4 and 1, 5, which match on 1, match on A, but do not match on B. So, I can quite easily say that this does not hold. But if you look at the column B, they have all distinct values.

So, if two tuples match on B, they should match on A. This I am just observing from the data. I do not know whether this is an actual rule. But I am just saying that we can, what we can confidently say from this is A determines B does not hold. We cannot confidently say whether B determines A should hold or not.

So, but if we assume that B determines A holds then we will say that, given this instance, we cannot have a tuple like 2, 4, because 4 is already there and it cannot have a different value of A given the 4, because I am assuming that B determines A. I cannot have a 3, 5, because I already have, I cannot have a 4, 7 like that.

So, if this holds, I know, these are the facts. Given the data I know A determines B cannot hold, but whether B determines A should hold is not to be deduced from the data. But it has to come as an external business rule in the design, which we will have to write down in terms of the functional dependency.

(Refer Slide Time: 11:07)



Now, these are some implications of that. Obviously, any set of attributes will be called a superkey if K functionally determines R. I mean, it is just the notion of key being told in a different way. What did we say that a superkey is one so that the values of attributes on the, on those, on the K attributes will uniquely identify a tuple, uniquely determine a tuple. So, which means that if we have two tuples which match on the K part, they have to match on the (())(11:46) of R. So, K functionally determines R.

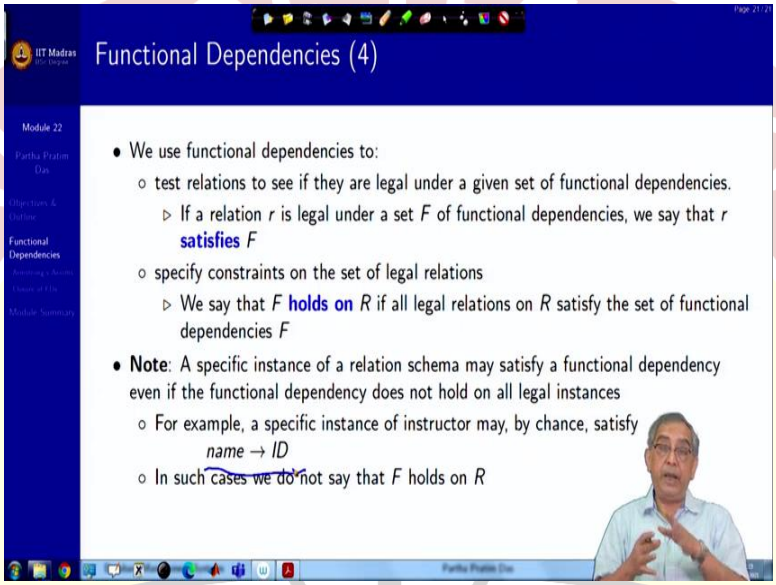
What is the candidate key? Say, if I have a K it is a candidate key if it determines R then it is a superkey and a candidate key is whose which is minimal, that which does not have a subset which is a superkey. So, which has no alpha subset of K such that alpha determines R. So, K determines R but none of its subsets determine R, so that is the definition of the candidate key.

So, you can say whatever we said in terms of kind of English statement, now, we are finding out a mathematical expression for them for the key we have, superkey we have found, for candidate key we have found and so on. So, given, just an example, given the relation, instructor with department, you have ID name all of that. So, we know we have already seen. There is a, I am sorry, there is a typo here. This should be read as budget. This is just copy paste error.

So, department name determines building, department name determines budget, ID determines building, because an employee or an instructor having an ID works for a department. So, it is kind of a transitive, logical relationship, but inside that, obviously, since every department has a

unique building, an instructor will sit in a unique building. Given the instructor there cannot be two buildings. So, ID will determine building and so on. There could be several more. But, for example, you would not expect the department name to decide on the salary. So, this is not a functional dependency that you can expect to hold. So, these are the basic.

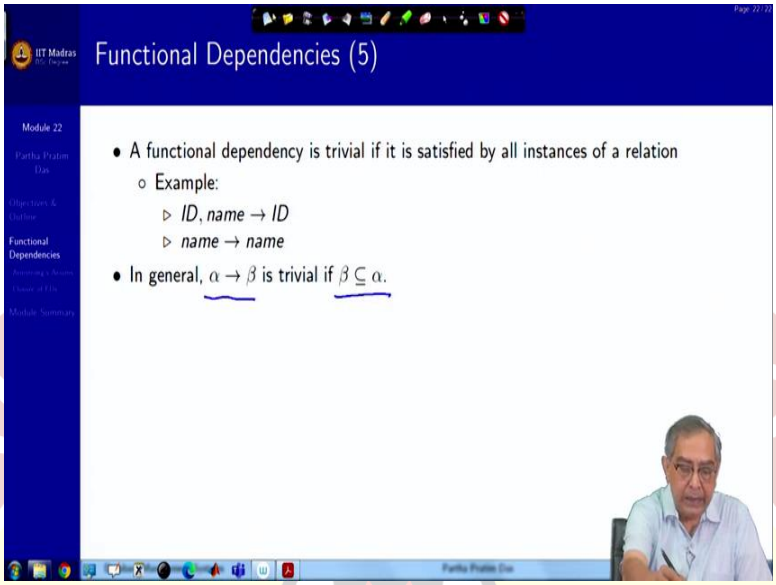
(Refer Slide Time: 14:03)



So, we say if for a relation schema, we say a set of functional dependencies. Now, we are not talking about one, we just saw that could be multiple. We say that a set of functional dependencies hold on a relation if every instance of the relational schema satisfy every functional dependency that involves in it.

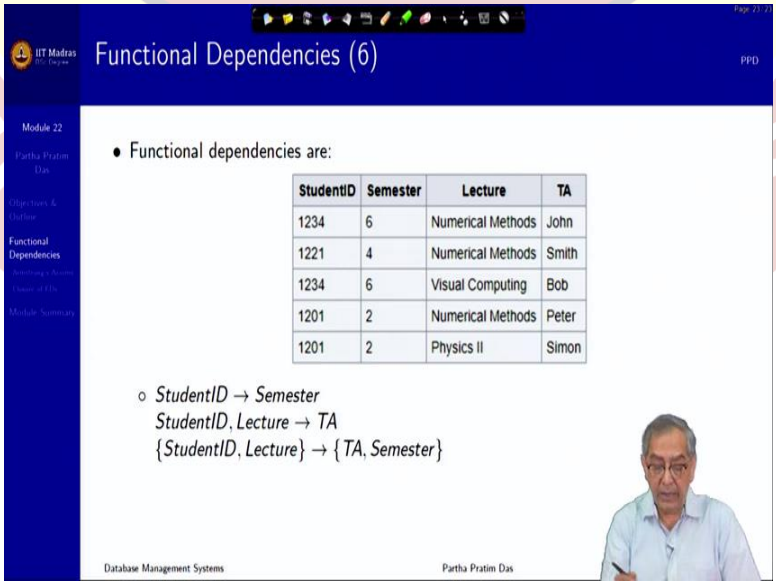
So, it is very important to note that it is a guiding principle. It is not, again, from the observation of the data. So, it may be you have in an instructor, instance of instructor table, many a times we will have that, where every instructor has a unique name. That does not mean that you can conclude that name, functionally determines ID. It is just a coincidence. To determine that name functionally determines ID you have to have the assumption that name cannot be duplicate, which is naturally not possible. So, keep this in mind in terms of functional dependencies.

(Refer Slide Time: 15:14)



Now, certain functional dependencies could be trivial. That if I say that ID name will always functionally determine ID, because if two tuples match on ID name, they obviously match on ID. Name will determine, I mean, in generality we can say that this is trivial if right hand side is a subset of the left hand side, because we are seeing that the tuple matching on the left hand side part will ensure that it matches on the right hand side part. So, if right hand side is a subset of the left hand side part, then it is trivially, trivial dependency.

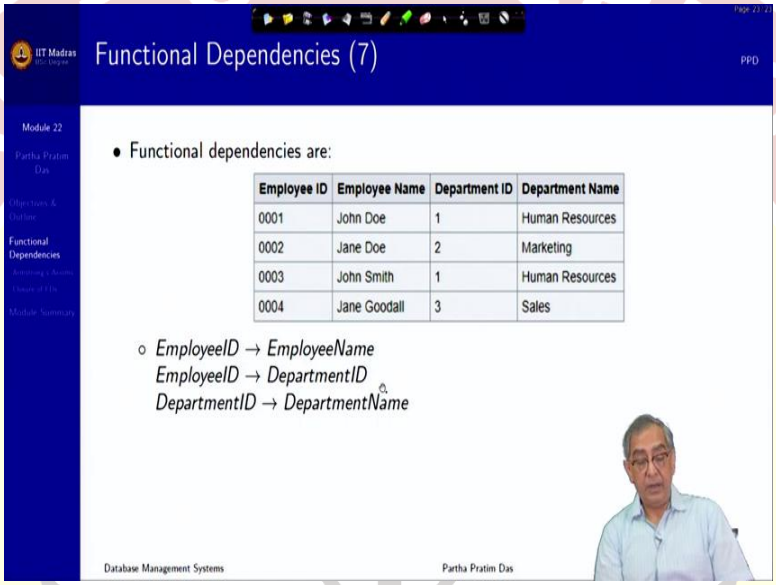
(Refer Slide Time: 15:50)



StudentID	Semester	Lecture	TA
1234	6	Numerical Methods	John
1221	4	Numerical Methods	Smith
1234	6	Visual Computing	Bob
1201	2	Numerical Methods	Peter
1201	2	Physics II	Simon

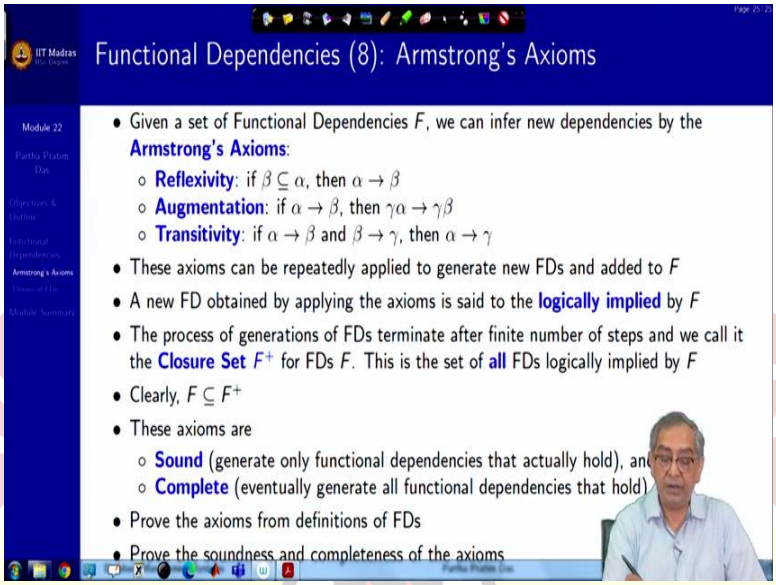
So, here are some examples. You can think over them more. For example, given a student ID in this student semester table, the semester will be known given a student ID and the lecture, the TA will be known. So, we can say that in generality, student ID lecture will determine TA semester. So, these are kind of inferences you can make and we are going to soon see what are the rules for making these inferences.

(Refer Slide Time: 16:19)



Similarly, here employee ID will always uniquely determine employee name, employee ID will uniquely determine employee, department ID, department ID will uniquely determine department name and so on.

(Refer Slide Time: 16:35)



So, there are a set of rules which guide the reasoning about functional dependencies. These are actually rules, but who wrote them called it them axioms. So, they are colloquially or they are, in the community they are referred to as axioms, but they are actually rules which can be proven. So, these rules are very natural, but needs to be known and remembered.

So, first is the reflexivity which we have already talked of. If the right hand side is a subset of the left hand side then you have a dependency which is reflexive, because it is leading to itself. What is reflexive, a relation which leads to itself. Then you say we have an augmentation. So, there is a functional dependency alpha determines beta and gamma is some set of attributes which may have something common with alpha, may have something common with beta, may not have anything common with any one of them whatever. If you augment them on two sides, then this functional dependency will also hold.

It is very simple to reason that, because if we say two tuples switch match on gamma alpha, then what you are saying, then they match on gamma beta. The fact that if the match on alpha, they will match on the beta part is the given functional dependency alpha determines beta. We have added the gamma set on left. We have added the gamma set on right. The left is saying that if they match on gamma and alpha, obviously, they match and gamma and from alpha determines beta they imagine beta. So, this will hold. These are easy to prove, but, so, I have not written the proof, but I will encourage you to do the proof at home.

Another is transitivity. If A determines, alpha determines beta, beta determines gamma then alpha determines gamma. Again, quite obviously, if alpha parts between the two tuples are same, their beta parts **will be same**. And if the beta parts are same, beta determines gamma says the gamma part will have to be same. So, I can club them together say if the alpha parts are same then the gamma parts will have to be same. So, these can be, these axioms can be repeatedly applied to a set of functional dependencies.

And we say when we apply and get a new functional dependency by applying them then we say that that is logically implied by the original set. And in that process, we can keep on adding more and more functional dependencies to the original set till we can add no more and that is called the closure set of a set of functional dependencies. This is a set of all possible functional dependencies that can be logically implied by the original set of functional dependencies. This is typically marked as F+. Obviously, F is a subset of F+. We will talk more about the closer set, do not, and we will have examples.

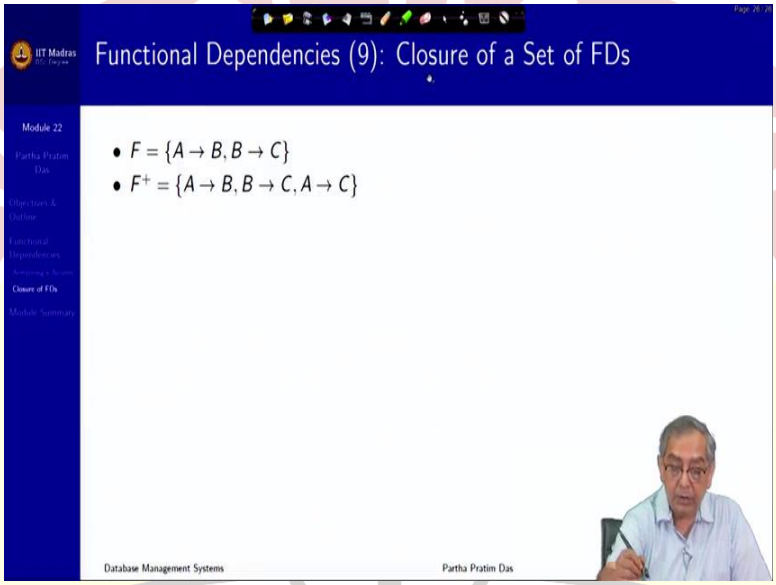
Now, two key factors about these axioms are, these are rules to compute to get more and more functional dependencies. So, we need two things. One is we need soundness. That is, if we infer a functional dependency **by this process**, by the Armstrong's axiom, it must be a dependency that actually holds. It has to be mathematically sound, which kind of is easy to prove, because all that you need to do is to prove that each one of the rules, each one of the axioms are sound.

The other one is a little trickier, which says that these axioms are complete. That is to say that, if I take a set of functional dependencies F and keep on applying these axioms, then I will eventually generate all functional dependencies that hold. I will never miss out anything. So, these are complete that by this process we will be able to get, you do not need any other rules or anything, but by this process, we will be able to get all the functional dependencies that hold on the, I mean, that can be logically implied from the given set F.

Now, completeness is little trickier to prove. All that you will need to do is to do some kind of a contradiction, say that it is not complete and you have a functional dependency that is not implied through the **applications of** Armstrong axiom. And then you have two cases, I mean, since you are saying that it is not implied from that, so obviously it is not in the original set F.

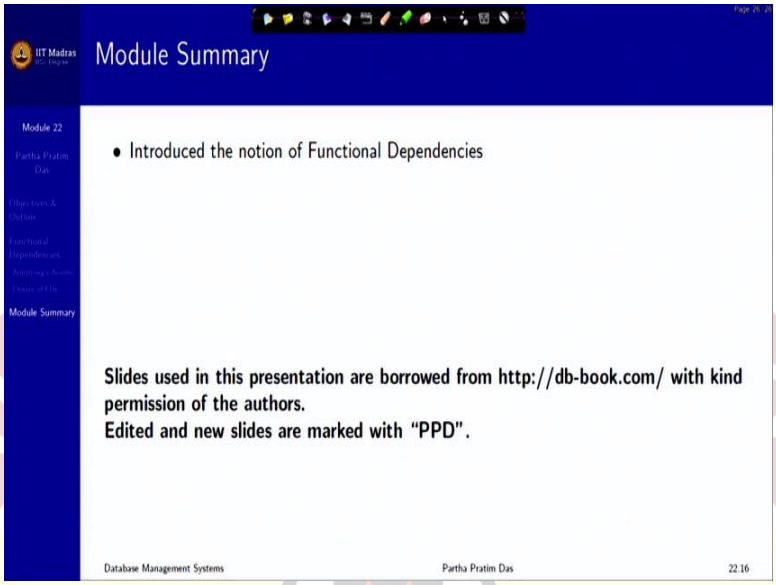
So, then you have to say that how could you, if it is not in the original set F and it still holds, then it must contradict one of these laws. So, think over this, work it out. So, prove these axioms from the definitions of functional dependencies and also prove the soundness and completeness of the axioms to understand them better.

(Refer Slide Time: 22:14)



So, this is just a simple example. F is A determines B and B determines C. So, F+ is I can use transitivity A determines B and B determines C. So, I have A determines C also. And you can see no other rule can be applied on this further. So, F+ or the closure of this F is A determines B, B determines C, A determines C.

(Refer Slide Time: 22:41)



We will have bigger examples in the next module and we will also talk about how to compute this subsequently. So, in this module, we have formulated the basic notion of functional dependencies and how does it help capture the real world constraints, the business rules that must exist and it is now giving us a formal language of mathematics on the relational algebra to talk about these constraints, to reason about them and to use them in the designing.

That process we have been able to formally define what is a superkey, formally define what is the candidate key. We will see that the implications of different types of relations and dependencies will now be possible to be expressed through these functional dependencies and subsequently through other advanced forms of dependencies. So, thank you very much friends for your attention, and we will close this module here. See you again in the next module.